Planificación, Ciclo de Vida y Gantt

Temporis:

Una herramienta accesible para el control y la optimización de nuestro tiempo

Kevin Zamora Amela

1. Planificación del Ciclo de Vida y Diagrama de Gantt

1. Objetivos y Alcance del Proyecto

El proyecto **Temporis** tiene como objetivo principal el desarrollo de una herramienta multiplataforma accesible para el control y la optimización del tiempo personal y profesional.

- **Objetivo General:** Diseñar y programar una aplicación que permita a los usuarios gestionar sus tareas y tiempos mediante temporizadores personalizables, garantizando la accesibilidad y la sincronización en la nube.
- **Alcance Funcional:** La aplicación incluirá un sistema de autenticación seguro (Firebase), gestión completa (Funciones "CRUD") de temporizadores, visualización de estadísticas de uso, personalización visual y ajustes "avanzados" de accesibilidad, para asegurar la inclusión de todos/as los/as usuarios.
- **Exclusiones** (Fuera de alcance): Para asegurar la viabilidad del proyecto en los plazos establecidos, quedan excluidos el desarrollo de una versión nativa para "smartwatches" y la integración con calendarios de terceros (Google/Outlook), al menos en esta primera fase.

2. Ciclo de Vida del *Software*

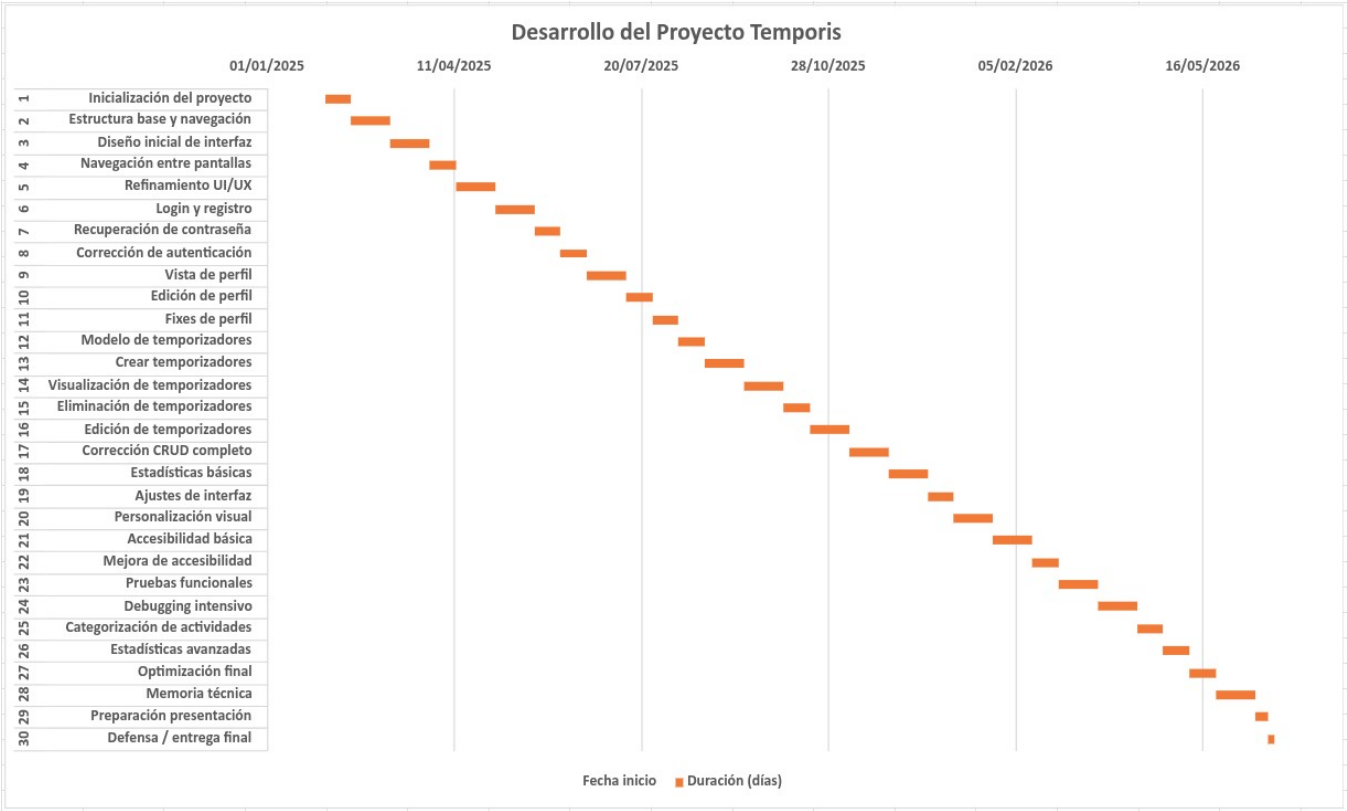
Para el desarrollo de **Temporis**, he seleccionado un **Modelo de Ciclo de Vida Incremental**.

- **Justificación:** Este modelo permite dividir el producto en distintas funcionalidades que se van integrando de forma progresiva. Dado que el proyecto requiere una base sólida de autenticación antes de gestionar datos sensibles, el enfoque incremental facilita la validación de cada módulo (perfil, temporizadores, estadísticas) de manera independiente.

  Se elige el modelo incremental porque permite mitigar riesgos técnicos de forma temprana. Por ejemplo, al completar el incremento de 'Autenticación con Firebase' en la Fase 2, aseguramos la persistencia de datos antes de desarrollar la lógica compleja de los temporizadores en la Fase 3.
- **Ventajas:** Reduce los riesgos de errores críticos al final del desarrollo y permite obtener versiones funcionales del "software" de forma temprana, lo cual es vital para el cumplimiento de los hitos académicos.

3. Planificación y Diagrama de Gantt

La planificación se ha estructurado en un total de **30 tareas** detalladas, cubriendo un periodo de ejecución que se extiende **desde febrero de 2025 hasta junio de 2026**.



3.1. Fases y Plazos de Ejecución

El cronograma se divide en bloques lógicos para garantizar un flujo de trabajo coherente:

- **Fase de Inicio y Diseño (Feb 2025 - Abr 2025):** Incluye la inicialización, diseño de interfaz y refinamiento UI/UX. Es la fase crítica para establecer la experiencia de usuario.
- **Fase de Desarrollo del Núcleo (Abr 2025 - Oct 2025)**: Implementación del sistema de usuarios, gestión de perfiles y la lógica principal de los temporizadores.
- **Fase de Análisis y Calidad (Nov 2025 - Mar 2026):** Periodo dedicado a la corrección del CRUD completo, estadísticas y, sobre todo, un bloque de 21 días de “Debugging” intensivo, para asegurar la estabilidad.
- **Fase de Cierre (Abr 2026 - Jun 2026):** Optimización final, redacción de la memoria técnica y preparación de la defensa pública.

Tareas definidas y representadas mediante el diagrama de Gantt:

Nº	Tarea	Fecha inicio	Duración (días)	Fecha fin
1	**Inicialización del proyecto**	01/02/2025	14	14/02/2025
2	**Estructura base y navegación**	15/02/2025	21	07/03/2025
3	**Diseño inicial de interfaz**	08/03/2025	21	28/03/2025
4	**Navegación entre pantallas**	29/03/2025	14	11/04/2025
5	**Refinamiento UI/UX**	12/04/2025	21	02/05/2025
6	**Login y registro**	03/05/2025	21	23/05/2025
7	**Recuperación de contraseña**	24/05/2025	14	06/06/2025
8	**Corrección de autenticación**	07/06/2025	14	20/06/2025
9	**Vista de perfil**	21/06/2025	21	11/07/2025
10	**Edición de perfil**	12/07/2025	14	25/07/2025
11	**Fixes de perfil**	26/07/2025	14	08/08/2025
12	**Modelo de temporizadores**	09/08/2025	14	22/08/2025
13	**Crear temporizadores**	23/08/2025	21	12/09/2025
14	**Visualización de temporizadores**	13/09/2025	21	03/10/2025
15	**Eliminación de temporizadores**	04/10/2025	14	17/10/2025
16	**Edición de temporizadores**	18/10/2025	21	07/11/2025
17	**Corrección 'CRUD' completo**	08/11/2025	21	28/11/2025
18	**Estadísticas básicas**	29/11/2025	21	19/12/2025
19	**Ajustes de interfaz**	20/12/2025	14	02/01/2026
20	**Personalización visual**	03/01/2026	21	23/01/2026
21	**Accesibilidad básica**	24/01/2026	21	13/02/2026
22	**Mejora de accesibilidad**	14/02/2026	14	27/02/2026
23	**Pruebas funcionales**	28/02/2026	21	20/03/2026
24	**'Debugging' intensivo**	21/03/2026	21	10/04/2026
25	**Categorización de actividades**	11/04/2026	14	24/04/2026
26	**Estadísticas avanzadas**	25/04/2026	14	08/05/2026
27	**Optimización final**	09/05/2026	14	22/05/2026
28	**Memoria técnica**	23/05/2026	21	12/06/2026
29	**Preparación presentación**	13/06/2026	7	19/06/2026
30	**Defensa / entrega final**	20/06/2026	3	22/06/2026

PD: *Las tareas "**coloreadas en rojo**" se han descartado por falta de tiempo y estas se implementarán presumiblemente en una "versión/release" siguiente de nuestra aplicación Temporis.*

3.2. Asignación de Recursos

Para el cumplimiento de los objetivos, se han previsto los siguientes recursos:

- **Recursos Humanos:** El proyecto es desarrollado de forma individual, asumiendo los roles de Analista, Diseñador UI/UX, Desarrollador *Frontend/Backend* y *QA Tester*.
- **Recursos Materiales:** Uso de *hardware* de desarrollo (PC/*Smartphone*), entornos de desarrollo (Android Studio y VS Code), servicios en la nube (Firebase para autenticación y base de datos) y herramientas de gestión (GitHub para control de versiones y ClickUp/Excel (para el "Gantt").
- **Recursos Económicos:** Se estima un presupuesto de 0€ en licencias de software al priorizar herramientas de código abierto (Android Studio y VS Code) y capas gratuitas de servicios *cloud* (*Firebase Free Tier* y GitHub).

4. Análisis de Riesgos y Control

El **Diagrama de Gantt y el Ciclo de Vida** se han diseñado aplicando márgenes de maniobra, sobretodo en las tareas de "Fixes (o Reparaciones)" y "Mejora de accesibilidad". Estas tareas nos permiten “absorber” posibles retrasos en la integración de “funciones complejas”, asegurando que la fecha de entrega final no se vea comprometida.